ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA



LG DUT CYBER SECURITY LAB

BÁO CÁO THỰC TẬP TUẦN 2

Thành viên Nhóm 4

1. Nguyễn Trường Bảo
2. Mai Thị Mỹ Đình
3. Bùi Phương Chi
4. Nguyễn Đình Nhất
5. Đỗ Hữu Dũng
6. Nguyễn Thành Lưu

Đà Nẵng, ngày ... tháng ... năm ...

Mục lục

4. Triển khai truyền thông giữa các thiết bị thông qua giao diện VLAN và IPv6 **1**
4.1. Cơ sở lí thuyết . . . 1
4.1.2 Ethernet . . . 1
4.1.3 Virtual LAN (VLAN) . . . 1
4.2. Địa chỉ IPv6 trong mạng . . . 2
4.2.1. Khái niệm . . . 2
4.2.2. Ứng dụng trong bài thực hành . . . 3
4.3. Công cụ Scapy . . . 3
4.3.1. Khái niệm . . . 3
4.3.2. Các chức năng . . . 3
4.4. Nftables . . . 4
4.4.1. Cấu hình nftables . . . 6

5. Thiết lập môi trường và xây dựng chương trình **9**
5.1. Môi trường . . . 9
5.1.1. Cấu hình Router . . . 9
5.1.2. Cấu hình IPv6 và VLAN trên Windows . . . 10
5.1.3. Cấu hình IPv6 và VLAN trên Raspberry Pi . . . 11
5.2. Chương trình . . . 13

6. Triển khai và đánh giá kết quả **15**
6.1. Kịch bản 1 – Kiểm tra kết nối và hiển thị thông tin giao diện mạng . . . 15
6.1.2. Thay đổi địa chỉ IPv6 . . . 16
6.1.3. Thay đổi VLAN ID . . . 18
6.1.4. Thay đổi địa chỉ MAC . . . 20
6.1.5. Thay đổi địa chỉ IPv4 . . . 22
6.2. Kịch bản 2 – Kiểm tra cơ chế chặn của tường lửa . . . 23
6.2.1. Kiểm tra với địa chỉ MAC không hợp lệ . . . 23
6.2.2. Kiểm tra với địa chỉ IPv6 không hợp lệ . . . 25
6.2.3. Kiểm tra với địa chỉ MAC và IPv6 đều không hợp lệ . . . 27

7. Kết luận **29**

4. Triển khai truyền thông giữa các thiết bị thông qua giao diện VLAN và IPv6

4.1. Cơ sở lí thuyết

4.1.2 Ethernet

Ethernet là công nghệ mạng có dây được sử dụng phổ biến trong các mạng cục bộ (LAN) để kết nối các thiết bị và trao đổi dữ liệu. Công nghệ này cung cấp một tập hợp các chuẩn truyền thông giúp các thiết bị trong cùng hệ thống có thể giao tiếp với nhau một cách ổn định, tin cậy và hiệu quả. Khác với Wi-Fi sử dụng sóng vô tuyến, Ethernet truyền dữ liệu thông qua cáp mạng, nhờ đó mang lại tốc độ truyền ổn định, độ trễ thấp, ít bị ảnh hưởng bởi nhiễu điện từ và hạn chế tình trạng mất gói tin. Những đặc điểm này đặc biệt phù hợp với các bài thực hành yêu cầu truyền và phân tích gói tin có độ chính xác cao, chẳng hạn như việc sử dụng Scapy để kiểm tra và đánh giá hoạt động của tường lửa giữa máy tính Windows và Raspberry Pi. Tuy nhiên, Ethernet cũng tồn tại một số hạn chế như yêu cầu hệ thống cáp vật lý, khó triển khai trên diện rộng và kém linh hoạt đối với các thiết bị cần tính di động cao.

Về mặt kỹ thuật, Ethernet hoạt động ở **lớp Liên kết dữ liệu (Data Link Layer – Layer 2)** trong mô hình tham chiếu OSI. Tại lớp này, các thiết bị được nhận diện thông qua **địa chỉ MAC (Media Access Control)** thay vì địa chỉ IP. Đây là một đặc điểm quan trọng trong quá trình thực hiện đề tài, bởi Scapy cho phép thao tác trực tiếp trên khung Ethernet (Ethernet Frame), từ đó người dùng có thể tùy chỉnh nhiều trường thông tin khác nhau để xây dựng các gói tin theo mục đích kiểm thử, bao gồm:

- Địa chỉ MAC nguồn (Source MAC Address) và MAC đích (Destination MAC Address).
- Trường Ethertype để xác định giao thức tầng trên.
- VLAN ID phục vụ kiểm thử mạng VLAN theo chuẩn IEEE 802.1Q.
- Địa chỉ IPv4 hoặc IPv6.
- Các giao thức tầng vận chuyển và tầng mạng như TCP, UDP, ICMP, ...

Sau khi hoàn tất việc thiết lập hạ tầng mạng, cấu hình VLAN và định tuyến IPv4/IPv6 giữa máy tính Windows và Raspberry Pi, hệ thống sẽ triển khai các chương trình Python sử dụng thư viện Scapy để xây dựng và gửi các gói tin tùy chỉnh (Custom Packets). Các gói tin này được truyền qua mạng nhằm mô phỏng nhiều tình huống giao tiếp khác nhau, từ đó quan sát phản ứng của tường lửa đối với từng loại lưu lượng. Kết quả thu được sẽ được sử dụng để đánh giá khả năng lọc gói tin, phát hiện lưu lượng bất thường và hiệu quả bảo vệ của hệ thống tường lửa trong môi trường thử nghiệm.

4.1.3 Virtual LAN (VLAN)

Virtual LAN (VLAN) là công nghệ cho phép phân chia một mạng LAN vật lý thành nhiều mạng logic độc lập. Mỗi VLAN hoạt động như một mạng LAN riêng biệt, giúp các thiết bị trong cùng một VLAN có thể giao tiếp với nhau như đang kết nối trong cùng một mạng vật lý, mặc dù chúng có thể được kết nối vào các cổng khác nhau của switch hoặc đặt tại những vị trí khác nhau trong hệ thống.

Về nguyên lý hoạt động, VLAN sử dụng **VLAN Tag** theo chuẩn **IEEE 802.1Q** để đánh dấu các khung Ethernet. Thẻ VLAN này chứa **VLAN ID**, cho phép các thiết bị mạng như switch và router xác định gói tin thuộc VLAN nào, từ đó chuyển tiếp dữ liệu đến đúng miền mạng tương ứng. Nhờ cơ chế gắn thẻ này, nhiều mạng logic có thể cùng sử dụng chung một hạ tầng mạng vật

lý mà vẫn đảm bảo sự tách biệt về lưu lượng.

Việc triển khai VLAN mang lại nhiều lợi ích như tăng cường khả năng quản lý hệ thống mạng, giảm lưu lượng quảng bá (Broadcast Traffic), nâng cao mức độ bảo mật thông qua việc cô lập các nhóm thiết bị và giúp việc mở rộng hoặc thay đổi cấu trúc mạng trở nên linh hoạt hơn. Thay vì phải thay đổi kết nối vật lý, người quản trị chỉ cần cấu hình lại VLAN trên các thiết bị mạng để di chuyển hoặc phân chia các nhóm người dùng.

Trong hệ thống mạng, mỗi VLAN được gán một **VLAN ID** duy nhất để phân biệt với các VLAN khác. Đây cũng là một trường quan trọng trong khung Ethernet mà thư viện **Scapy** có thể tạo hoặc chỉnh sửa trực tiếp. Trong phạm vi đề tài, VLAN ID sẽ được sử dụng để xây dựng các gói tin tùy chỉnh nhằm kiểm tra quá trình xử lý lưu lượng của hệ thống mạng và đánh giá khả năng hoạt động của tường lửa khi tiếp nhận các gói tin thuộc nhiều VLAN khác nhau.

4.2. Địa chỉ IPv6 trong mạng

4.2.1. Khái niệm

IPv6 (Internet Protocol version 6) là phiên bản mới của giao thức Internet (IP), được phát triển nhằm khắc phục những hạn chế của IPv4, đặc biệt là tình trạng cạn kiệt không gian địa chỉ. Trong khi IPv4 sử dụng địa chỉ có độ dài **32 bit**, chỉ cung cấp khoảng **4,3 tỷ địa chỉ**, thì sự phát triển mạnh mẽ của Internet, điện toán đám mây, thiết bị di động và Internet of Things (IoT) đã khiến số lượng địa chỉ này không còn đáp ứng đủ nhu cầu kết nối. Để giải quyết vấn đề đó, IPv6 được chuẩn hóa với không gian địa chỉ lớn hơn rất nhiều.

IPv6 sử dụng địa chỉ có độ dài **128 bit**, cung cấp số lượng địa chỉ gần như vô hạn đối với nhu cầu hiện nay và trong tương lai. Không chỉ mở rộng không gian địa chỉ, IPv6 còn cải thiện hiệu quả định tuyến, hỗ trợ cấu hình địa chỉ tự động (Stateless Address Autoconfiguration – SLAAC), tích hợp tốt hơn với các cơ chế bảo mật và đơn giản hóa cấu trúc của gói tin so với IPv4.

Địa chỉ IPv6 được biểu diễn dưới dạng **8 nhóm số hệ thập lục phân**, mỗi nhóm gồm **4 ký tự**, các nhóm được phân tách bằng dấu hai chấm (:). Ví dụ:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

Trong thực tế, IPv6 cho phép rút gọn địa chỉ bằng cách:

- Loại bỏ các số **0** ở đầu mỗi nhóm.
- Thay một dãy các nhóm có giá trị **0000** liên tiếp bằng ký hiệu **::** (chỉ được sử dụng một lần trong mỗi địa chỉ).

Ví dụ, địa chỉ trên có thể được viết gọn thành:

```
2001:db8:85a3::8a2e:370:7334
```

Một địa chỉ IPv6 thông thường gồm ba thành phần chính:

- **Global Prefix (48 bit):** Xác định mạng hoặc tổ chức được cấp phát địa chỉ.
- **Subnet ID (16 bit):** Xác định mạng con bên trong hệ thống.
- **Interface ID (64 bit):** Xác định duy nhất giao diện mạng của từng thiết bị.

Trong phạm vi đề tài, giao thức IPv6 đóng vai trò là nền tảng truyền thông giữa máy tính Windows và Raspberry Pi. Thông qua thư viện **Scapy**, các trường thông tin của gói tin IPv6 như

địa chỉ nguồn, địa chỉ đích và các trường trong tiêu đề gói tin có thể được tùy chỉnh để tạo ra nhiều kịch bản kiểm thử khác nhau, phục vụ việc đánh giá khả năng xử lý và lọc lưu lượng của hệ thống tường lửa.

4.2.2. Ứng dụng trong bài thực hành

Trong bài thực hành, IPv6 được sử dụng để thiết lập kết nối trực tiếp giữa máy tính Windows và Raspberry Pi nhằm kiểm tra quá trình truyền nhận dữ liệu cũng như đánh giá hoạt động của tường lửa trên Raspberry Pi thông qua các gói tin UDP được tạo bằng Scapy.

Hai thiết bị được cấu hình địa chỉ IPv6 như sau:

- **Laptop (Windows):** fd53:1234:5678:1::20/64
- **Raspberry Pi:** fd53:1234:5678:1::10/64

Cả hai địa chỉ đều thuộc cùng một mạng con **fd53:1234:5678:1::/64**, do đó hai thiết bị có thể giao tiếp trực tiếp mà không cần thông qua bộ định tuyến.

Sau khi hoàn tất cấu hình mạng, chương trình Python sử dụng thư viện **Scapy** sẽ tạo và gửi các gói tin IPv6 tùy chỉnh. Trong quá trình này, có thể thay đổi nhiều trường thông tin của gói tin như địa chỉ IPv6 nguồn và đích, VLAN ID, giao thức tầng vận chuyển (UDP, TCP hoặc ICMPv6) và các trường khác trong tiêu đề gói tin. Những gói tin được xây dựng sẽ được gửi tới Raspberry Pi để quan sát phản ứng của hệ thống và đánh giá khả năng phát hiện, xử lý cũng như cho phép hoặc chặn lưu lượng của tường lửa theo từng kịch bản kiểm thử.

4.3. Công cụ Scapy

4.3.1. Khái niệm

- **Scapy** là một thư viện mã nguồn mở mạnh mẽ của Python, được sử dụng để tạo (craft), gửi (send), nhận (receive) và phân tích (analyze) các gói tin mạng một cách linh hoạt.
- Scapy hỗ trợ nhiều giao thức mạng phổ biến như **Ethernet, ARP, IPv4, IPv6, TCP, UDP, ICMP, DNS, DHCP** và nhiều giao thức khác. Công cụ này cho phép người dùng tùy chỉnh từng trường (field) của gói tin ở các tầng của mô hình TCP/IP, từ tầng liên kết dữ liệu (Data Link Layer) đến tầng giao vận (Transport Layer), thậm chí có thể mở rộng sang các giao thức ở tầng ứng dụng.
- Nhờ khả năng thao tác trực tiếp trên cấu trúc gói tin, Scapy được sử dụng rộng rãi trong các lĩnh vực như kiểm thử bảo mật (Security Testing), kiểm thử xâm nhập (Penetration Testing), phân tích lưu lượng mạng (Network Traffic Analysis), kiểm tra hoạt động của các giao thức và mô phỏng nhiều kiểu tấn công mạng nhằm đánh giá khả năng phòng vệ của hệ thống.
- Trong bài thực hành này, Scapy được sử dụng để xây dựng và gửi các gói tin tùy chỉnh từ máy **Windows** đến **Raspberry Pi**. Thông qua việc thay đổi các trường của gói tin và quan sát phản hồi từ hệ thống, nhóm tiến hành kiểm tra hoạt động của tường lửa (Firewall), đánh giá khả năng lọc gói tin và xác minh việc xử lý lưu lượng mạng trong các kịch bản khác nhau.

4.3.2. Các chức năng

Scapy là một công cụ mạnh mẽ, cung cấp nhiều chức năng phục vụ cho việc nghiên cứu, kiểm thử và phân tích mạng. Một số chức năng tiêu biểu gồm:

- **Tạo gói tin (Packet Crafting):**

Scapy cho phép tạo các gói tin từ nhiều giao thức khác nhau như **Ethernet, ARP, IPv4, IPv6, TCP, UDP, ICMP** và nhiều giao thức khác. Người dùng có thể tùy chỉnh hầu hết các trường (field) trong gói tin, chẳng hạn như địa chỉ MAC nguồn/đích, địa chỉ IP nguồn/đích, VLAN ID, cổng nguồn/đích, TTL, cờ TCP (TCP Flags), dữ liệu tải (Payload), ... Điều này giúp mô phỏng nhiều tình huống truyền thông hoặc kiểm thử bảo mật khác nhau.

- **Gửi và nhận gói tin (Packet Sending and Receiving):**
  Scapy hỗ trợ gửi các gói tin tùy chỉnh đến thiết bị đích và nhận phản hồi tương ứng. Công cụ có thể gửi từng gói tin riêng lẻ hoặc gửi nhiều gói tin liên tục để kiểm tra khả năng xử lý của hệ thống, đánh giá hiệu năng mạng hoặc mô phỏng các kịch bản truyền thông thực tế.
- **Phân tích gói tin (Packet Analysis):**
  Scapy có khả năng hiển thị chi tiết cấu trúc của từng gói tin, bao gồm các trường thông tin của từng lớp giao thức. Ngoài ra, Scapy có thể đọc và ghi các tệp định dạng **PCAP**, cho phép kết hợp với **Wireshark** để phân tích lưu lượng mạng một cách trực quan và chuyên sâu hơn.
- **Bắt gói tin (Packet Sniffing):**
  Scapy tích hợp chức năng bắt và giám sát lưu lượng mạng theo thời gian thực. Người dùng có thể áp dụng các bộ lọc theo địa chỉ IP, giao thức, cổng hoặc các điều kiện khác để chỉ thu thập những gói tin cần thiết, phục vụ cho việc giám sát, phân tích hoặc gỡ lỗi mạng.
- **Mô phỏng và kiểm thử bảo mật (Security Testing):**
  Nhờ khả năng tạo các gói tin tùy chỉnh, Scapy được sử dụng để mô phỏng nhiều tình huống kiểm thử bảo mật như giả mạo địa chỉ IP (IP Spoofing), giả mạo địa chỉ MAC (MAC Spoofing), gửi các gói tin bất thường hoặc tạo lưu lượng lớn nhằm đánh giá khả năng phát hiện và ngăn chặn của tường lửa (Firewall), hệ thống IDS/IPS và các cơ chế bảo vệ mạng khác.

4.4. Nftables

Nftables là framework quản lý và lọc gói tin hiện đại của Linux, được tích hợp trực tiếp trong nhân (Linux Kernel). Đây là hệ thống thay thế cho các công cụ truyền thống như **iptables**, **ip6tables**, **arptables** và **ebtables**, với mục tiêu cung cấp một cơ chế thống nhất, linh hoạt và hiệu quả hơn trong việc quản lý lưu lượng mạng.

Không chỉ là một công cụ cấu hình tường lửa, nftables còn là một kiến trúc xử lý gói tin cho phép người quản trị xây dựng các quy tắc (Rules) để kiểm soát, chuyển tiếp, ghi nhật ký (Logging), đánh dấu (Marking), dịch địa chỉ mạng (NAT) và thực hiện nhiều tác vụ khác đối với lưu lượng mạng.

Một ưu điểm nổi bật của nftables là sử dụng một cú pháp thống nhất cho cả IPv4, IPv6, ARP và Bridge, giúp đơn giản hóa việc quản lý và giảm số lượng quy tắc cần cấu hình so với iptables. Đồng thời, nftables hỗ trợ các cấu trúc dữ liệu như **Set**, **Map** và **Concatenation**, giúp tăng hiệu năng xử lý khi làm việc với số lượng lớn quy tắc.

Để hiểu rõ cách nftables hoạt động, cần nắm được rằng mọi gói tin khi đi qua hệ thống Linux đều sẽ đi qua các **Hook** (điểm móc) trong nhân Linux. Tại mỗi Hook, nftables có thể kiểm tra thông tin của gói tin và quyết định cho phép (Accept), từ chối (Drop), ghi nhật ký (Log), chuyển tiếp (Forward) hoặc thực hiện các hành động khác theo các quy tắc đã được cấu hình.

Các tầng hoạt động của nftables

Nftables có khả năng kiểm soát lưu lượng mạng ở nhiều tầng của mô hình TCP/IP (tương ứng với các tầng trong mô hình OSI). Tùy thuộc vào **address family** và **hook** được sử dụng, nftables có thể kiểm tra và xử lý gói tin ở các tầng sau:

- **1. Tầng Liên kết dữ liệu (Data Link Layer – Tầng 2):**

Ở tầng này, nftables xử lý trực tiếp các khung Ethernet thông qua **bridge family** hoặc **arp family**. Điều này cho phép lọc lưu lượng dựa trên các thông tin của tầng liên kết dữ liệu như:

- Địa chỉ MAC nguồn và MAC đích.
- Loại khung Ethernet (EtherType).
- Các gói tin ARP.

Ví dụ: Chặn một thiết bị dựa trên địa chỉ MAC hoặc kiểm soát lưu lượng đi qua một Linux Bridge.

Ứng dụng: Thích hợp cho các hệ thống sử dụng Bridge, môi trường ảo hóa, mạng LAN hoặc khi cần loại bỏ lưu lượng ngay từ tầng liên kết dữ liệu trước khi gói tin được xử lý ở tầng IP.

- **2. Tầng Mạng (Network Layer – Tầng 3):**

Đây là tầng được sử dụng phổ biến nhất trong nftables. Tại đây, nftables xử lý các gói tin IPv4 và IPv6 dựa trên các trường của tiêu đề IP, bao gồm:

- Địa chỉ IP nguồn.
- Địa chỉ IP đích.
- Giao thức tầng trên (TCP, UDP, ICMP, ICMPv6, . . .).
- Các trường khác trong tiêu đề IP như TTL, Hop Limit hoặc Traffic Class.

Ví dụ: Chặn một địa chỉ IPv6, chỉ cho phép các thiết bị thuộc một dải mạng truy cập hoặc cấu hình NAT và định tuyến giữa các mạng.

Ứng dụng: Xây dựng tường lửa, kiểm soát truy cập mạng, định tuyến và dịch địa chỉ mạng (NAT).

- **3. Tầng Giao vận (Transport Layer – Tầng 4):**

Sau khi xác định giao thức TCP hoặc UDP, nftables có thể lọc lưu lượng dựa trên:

- Cổng nguồn (Source Port).
- Cổng đích (Destination Port).
- Các cờ TCP (SYN, ACK, FIN, RST, . . .).
- Trạng thái kết nối thông qua **Connection Tracking (conntrack)**.

Ví dụ: Chỉ cho phép truy cập dịch vụ Web qua cổng 80 và 443, chặn truy cập SSH trên cổng 22 hoặc chỉ cho phép các kết nối đã được thiết lập (Established Connections).

Ứng dụng: Bảo vệ máy chủ, kiểm soát các dịch vụ mạng và quản lý các phiên kết nối một cách hiệu quả.

Vì sao lựa chọn nftables?

Trước đây, Linux sử dụng nhiều công cụ khác nhau để quản lý tường lửa như **iptables**, **ip6tables**, **arptables** và **ebtables**, trong đó mỗi công cụ chỉ hỗ trợ một nhóm giao thức hoặc một tầng mạng nhất định. Điều này khiến việc cấu hình và quản lý trở nên phức tạp.

Nftables được phát triển nhằm thay thế các công cụ trên bằng một framework thống nhất, mang lại nhiều ưu điểm như:

- Hỗ trợ đồng thời **IPv4, IPv6, ARP và Bridge** trong cùng một hệ thống quản lý.
- Có thể lọc lưu lượng từ **tầng liên kết dữ liệu (Layer 2)** đến **tầng giao vận (Layer 4)**.
- Cho phép lọc theo địa chỉ MAC, địa chỉ IP, giao thức, cổng, trạng thái kết nối và nhiều trường thông tin khác của gói tin.
- Hỗ trợ các cấu trúc dữ liệu như **Set**, **Map** và **Concatenation**, giúp giảm số lượng quy tắc và nâng cao hiệu năng xử lý.
- Cú pháp thống nhất, dễ quản lý và có hiệu suất cao hơn so với hệ thống iptables truyền thống.

Trong bài thực hành này, **nftables** được lựa chọn để xây dựng tường lửa trên Raspberry Pi nhằm kiểm tra khả năng lọc gói tin được tạo bởi **Scapy**. Việc sử dụng nftables giúp triển khai các quy tắc kiểm soát theo nhiều tiêu chí khác nhau như địa chỉ MAC, địa chỉ IPv6, giao thức và cổng dịch vụ, từ đó đánh giá toàn diện khả năng hoạt động của hệ thống tường lửa.

4.4.1. Cấu hình nftables

Để sử dụng **nftables** trên hệ điều hành **webOS OSE** chạy trên Raspberry Pi, cần bổ sung các thành phần liên quan ngay trong quá trình xây dựng (build) hệ điều hành. Theo mặc định, webOS OSE không tích hợp đầy đủ các module của **nftables** trong nhân Linux cũng như các gói phần mềm cần thiết trong hệ thống tập tin (Root File System). Vì vậy, cần chỉnh sửa cấu hình của kernel và image trước khi tiến hành build lại hệ điều hành.

Sau khi hoàn tất các thay đổi, hệ điều hành sẽ được biên dịch (Build) lại và nạp (Flash) vào Raspberry Pi để các tính năng của nftables có thể hoạt động.

Cấu hình Kernel

Đầu tiên, cần bổ sung các module của nftables vào cấu hình nhân Linux.

Truy cập thư mục:

```
/build-webos/meta-webososе/meta-webos-raspberrypi/
recipes-kernel/linux/linux-raspberrypi/
```

Mở tệp **bridge.cfg** và thêm các dòng sau:

```
CONFIG_NF_TABLES_BRIDGE=y
CONFIG_NFT_BRIDGE_META=y
CONFIG_NFT_BRIDGE_PAYLOAD=y
CONFIG_NFT_NAT=y
```

Ý nghĩa của từng tùy chọn như sau:

- **CONFIG_NF_TABLES_BRIDGE=y**
  Kích hoạt framework nftables cho **Bridge Family**, cho phép nftables xử lý các khung Ethernet tại tầng liên kết dữ liệu (Layer 2).
- **CONFIG_NFT_BRIDGE_META=y**
  Cho phép sử dụng các trường thông tin (Metadata) của bridge trong các quy tắc nftables, chẳng hạn như tên cổng mạng (Interface), VLAN hoặc các thuộc tính của khung Ethernet.
- **CONFIG_NFT_BRIDGE_PAYLOAD=y**
  Cho phép nftables truy cập và kiểm tra nội dung của các trường trong khung Ethernet hoặc gói tin khi làm việc với Bridge Family.
- **CONFIG_NFT_NAT=y**

Kích hoạt chức năng **Network Address Translation (NAT)** trong nftables, phục vụ cho các kịch bản chuyển đổi địa chỉ mạng nếu cần.

Sau khi thêm các tùy chọn trên, các module tương ứng sẽ được biên dịch cùng với nhân Linux trong quá trình build hệ điều hành.

Cập nhật Image của webOS

Bên cạnh việc kích hoạt các module trong kernel, cần bổ sung các gói phần mềm cần thiết vào Root File System để có thể sử dụng nftables sau khi hệ điều hành khởi động.

Truy cập thư mục:

```
/build-webos/meta-webosose/meta-webos/recipes-core/images
```

Mở tệp **webos-image.bb** và thêm đoạn cấu hình sau:

```
WEBOS_IMAGE_EXTRA_INSTALL:append = " \
htop \
tcpdump \
nftables \
kernel-module-xt-hashlimit \
ipset \
apt \
"
```

Trong đó:

- **htop:** Công cụ giám sát tài nguyên hệ thống theo thời gian thực.
- **tcpdump:** Công cụ bắt và phân tích lưu lượng mạng, được sử dụng để kiểm tra các gói tin trong quá trình thực nghiệm.
- **nftables:** Bộ công cụ quản lý và cấu hình tường lửa dựa trên framework nftables.
- **kernel-module-xt-hashlimit:** Module hỗ trợ giới hạn tốc độ gói tin (Rate Limiting), thường được sử dụng để giảm thiểu các cuộc tấn công như DoS hoặc ICMP Flood.
- **ipset:** Công cụ quản lý tập hợp địa chỉ IP hoặc cổng mạng, giúp xây dựng các quy tắc tường lửa hiệu quả hơn khi xử lý số lượng lớn địa chỉ.
- **apt:** Trình quản lý gói được bổ sung để thuận tiện cho việc cài đặt hoặc cập nhật một số phần mềm trong quá trình thử nghiệm (nếu image hỗ trợ).

Sau khi hoàn tất các thay đổi đối với kernel và image, tiến hành build lại hệ điều hành bằng hệ thống Yocto của webOS OSE. Image sau khi được biên dịch thành công sẽ được nạp (Flash) lên Raspberry Pi. Khi hệ thống khởi động, các module của nftables và các công cụ liên quan sẽ sẵn sàng để cấu hình và kiểm thử tường lửa trong các bài thực hành tiếp theo.

Danh sách các gói phần mềm trên được bổ sung nhằm cung cấp các công cụ cần thiết phục vụ cho việc giám sát hệ thống, phân tích lưu lượng mạng, cấu hình tường lửa và thực hiện các bài thực hành kiểm thử bảo mật.

Build lại hệ thống

Sau khi hoàn tất việc chỉnh sửa cấu hình của kernel và image, tiến hành biên dịch lại hệ điều hành bằng hệ thống build của Yocto.

Thực hiện lần lượt các lệnh sau:

```
bitbake virtual/kernel -c cleanall
```

```
bitbake virtual/kernel
bitbake webos-image
```

Ý nghĩa của từng lệnh như sau:

- bitbake virtual/kernel -c cleanall: Xóa toàn bộ kết quả build trước đó của kernel, bao gồm các tệp tạm và bộ nhớ đệm, nhằm đảm bảo các thay đổi trong cấu hình được áp dụng hoàn toàn.
- bitbake virtual/kernel: Biên dịch lại nhân Linux với các tùy chọn và module mới đã được bổ sung.
- bitbake webos-image: Xây dựng lại image của webOS OSE, đồng thời tích hợp kernel mới và các gói phần mềm đã được thêm vào hệ thống.

Sau khi quá trình build hoàn tất, image mới sẽ chứa đầy đủ các module của nftables cùng các công cụ hỗ trợ cần thiết cho việc cấu hình và kiểm thử tường lửa.

Flash lại webOS

Image sau khi được build thành công sẽ được ghi (Flash) lên thẻ nhớ hoặc thiết bị lưu trữ sử dụng cho Raspberry Pi theo quy trình cài đặt thông thường của webOS OSE.

Sau khi Raspberry Pi khởi động thành công với image mới, kiểm tra sự hiện diện của nftables bằng lệnh:

```
nft --version
```

Nếu lệnh trả về phiên bản của nftables, đồng thời các module liên quan đã được nạp thành công, hệ thống đã sẵn sàng để xây dựng các quy tắc tường lửa và thực hiện các bài thực hành kiểm thử bằng Scapy.

5. Thiết lập môi trường và xây dựng chương trình

5.1. Môi trường

5.1.1. Cấu hình Router

Cấu hình VLAN trên hệ thống

Trong mô hình thực nghiệm, các thiết bị được phân chia thành các mạng logic bằng công nghệ **Virtual Local Area Network (VLAN)** nhằm cô lập lưu lượng mạng và thuận tiện cho việc kiểm thử các quy tắc của tường lửa.

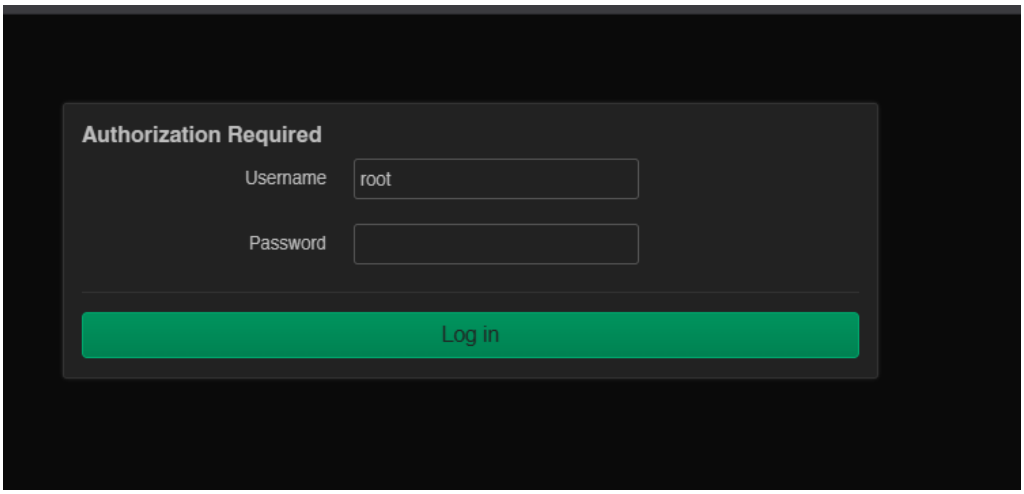
Để triển khai VLAN, router hoặc switch sử dụng trong hệ thống cần hỗ trợ chuẩn **IEEE 802.1Q (VLAN Tagging)**. Các bước cấu hình cơ bản được thực hiện như sau:

- **Bước 1: Kết nối đến thiết bị mạng**

Kết nối máy quản trị (Windows hoặc Linux) với router thông qua cáp mạng hoặc mạng Wi-Fi. Đảm bảo máy tính nhận được địa chỉ IP trong cùng mạng quản trị để có thể truy cập giao diện cấu hình.

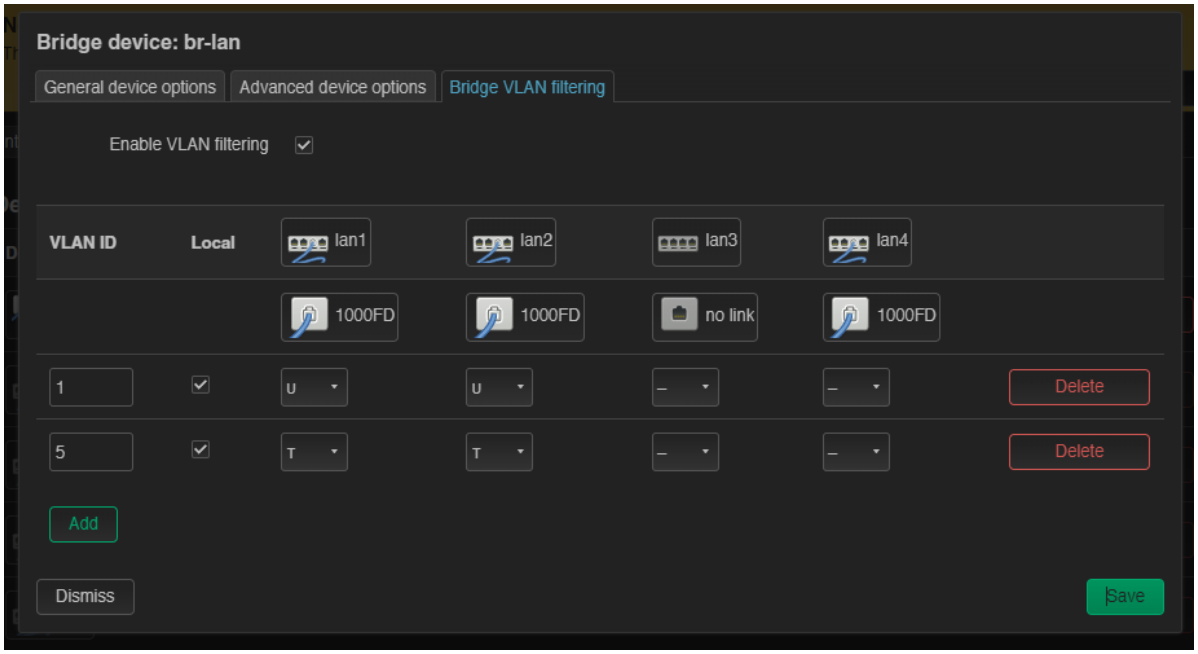
- **Bước 2: Truy cập giao diện quản trị**

Mở trình duyệt web và truy cập địa chỉ IP quản lý của router. Sau đó đăng nhập bằng tài khoản quản trị để thực hiện cấu hình.



- **Bước 3: Tạo VLAN**

Trên giao diện quản trị, tạo các VLAN cần sử dụng và chỉ định **VLAN ID** tương ứng. Trong bài thực hành này, hệ thống sử dụng **VLAN ID 5** để kết nối Raspberry Pi và các máy tính tham gia quá trình kiểm thử.



- **Bước 4: Gán cổng mạng vào VLAN**

Cấu hình các cổng Ethernet của router hoặc switch tham gia VLAN. Tùy theo mô hình triển khai, mỗi cổng có thể được cấu hình ở chế độ:

- **Access (Untagged):** Dành cho các thiết bị đầu cuối không hỗ trợ VLAN. Switch sẽ tự động thêm hoặc loại bỏ VLAN Tag khi truyền dữ liệu.
- **Trunk (Tagged):** Dành cho các liên kết giữa router và switch hoặc giữa các thiết bị hỗ trợ VLAN, cho phép truyền đồng thời nhiều VLAN trên cùng một đường kết nối.
- **Bước 5: Kiểm tra kết nối**

Sau khi hoàn tất cấu hình, kiểm tra khả năng giao tiếp giữa các thiết bị bằng các công cụ như **ping**, **tcpdump** hoặc **Scapy**. Các thiết bị thuộc cùng VLAN phải có khả năng liên lạc với nhau, trong khi lưu lượng giữa các VLAN khác nhau sẽ bị cô lập nếu chưa cấu hình định tuyến liên VLAN (Inter-VLAN Routing).

Việc cấu hình VLAN giúp tạo ra một môi trường mạng độc lập để thực hiện các bài thực hành về IPv6, Scapy và nftables, đồng thời hạn chế ảnh hưởng đến các thiết bị khác trong cùng hệ thống mạng.

5.1.2. Cấu hình IPv6 và VLAN trên Windows

Sau khi kết nối máy tính Windows với thiết bị mạng thông qua cáp Ethernet, tiến hành cấu hình địa chỉ IPv6 theo các bước sau:

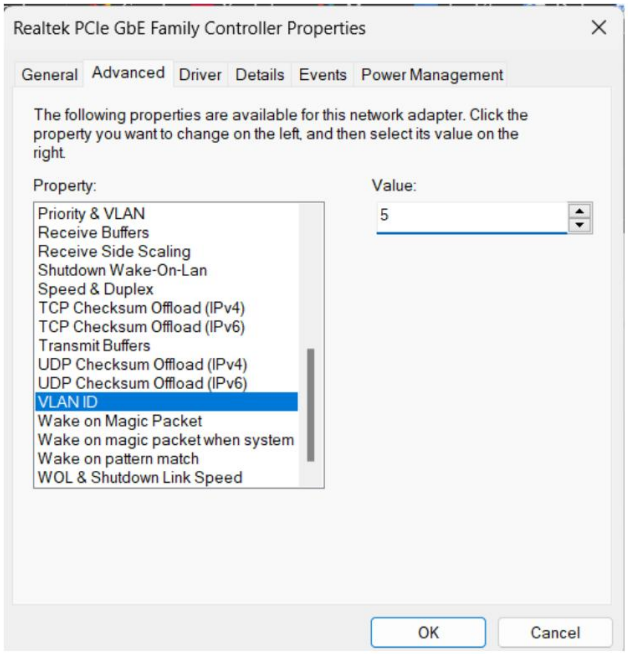
1. Mở **Control Panel**.
2. Chọn **Network and Internet** → **Network and Sharing Center**.
3. Chọn **Change adapter settings**.
4. Nhấp chuột phải vào **Ethernet** và chọn **Properties**.
5. Chọn **Internet Protocol Version 6 (TCP/IPv6)**, sau đó nhấn **Properties**.
6. Chọn **Use the following IPv6 address** và nhập địa chỉ IPv6, độ dài tiền tố (Prefix Length) và cổng mặc định (Default Gateway) nếu cần.

7. Nhấn **OK** để lưu cấu hình.

Trong phạm vi đề tài, máy tính Windows được cấu hình với địa chỉ IPv6:

- **IPv6 Address:** fd53:1234:5678:5::20
- **Prefix Length:** 64

Sau khi hoàn tất cấu hình IPv6, tiến hành cấu hình VLAN cho card mạng Ethernet bằng phần mềm hoặc trình điều khiển (driver) hỗ trợ chuẩn **IEEE 802.1Q**. Trong bài thực hành, card mạng được gán **VLAN ID = 5** để các gói tin gửi từ Windows được đánh dấu (tagged) thuộc VLAN 5 trước khi truyền đến Raspberry Pi và hệ thống mạng. Việc cấu hình VLAN giúp bảo đảm các gói tin được chuyển tiếp trong đúng miền mạng, đồng thời tạo điều kiện cho quá trình kiểm thử và đánh giá hoạt động của tường lửa với các gói tin thuộc nhiều VLAN khác nhau.



5.1.3. Cấu hình IPv6 và VLAN trên Raspberry Pi

Để Raspberry Pi có thể giao tiếp trong mạng IPv6 thông qua VLAN, trước tiên cần tạo một giao diện VLAN trên cổng Ethernet vật lý, sau đó gán địa chỉ IPv6 cho giao diện này. Trên hệ điều hành Linux, việc cấu hình VLAN có thể được thực hiện trực tiếp bằng dòng lệnh thông qua các công cụ quản lý mạng.

Bước 1. Kích hoạt hỗ trợ VLAN (IEEE 802.1Q)

Linux sử dụng module **8021q** để hỗ trợ chuẩn **IEEE 802.1Q**, cho phép hệ điều hành xử lý các khung Ethernet có gắn VLAN Tag. Nếu module chưa được nạp, có thể kích hoạt bằng lệnh:

```
sudo modprobe 8021q
```

Sau khi kích hoạt, kiểm tra các giao diện mạng hiện có để xác định cổng Ethernet sẽ sử dụng cho việc cấu hình mạng VLAN theo lệnh sau:

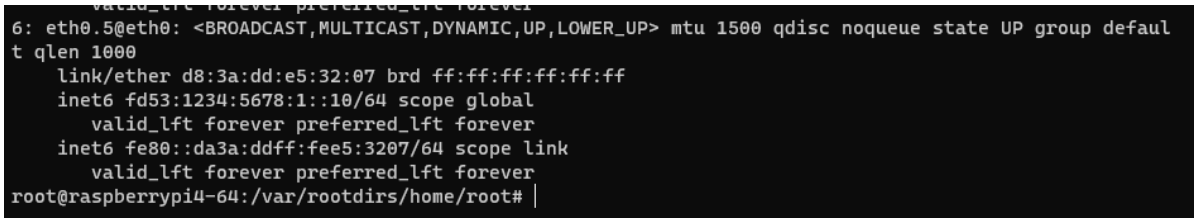
```
ip link show
```

Trong bài thực hành, giao diện Ethernet được sử dụng là **eth0**.

Bước 2. Tạo giao diện VLAN

Tiến hành tạo giao diện VLAN với **VLAN ID = 5** trên cổng Ethernet **eth0**, sau đó kích hoạt giao diện vừa tạo.

STT	Câu lệnh	Ý nghĩa
1	`ip link add link eth0 name eth0.5 type vlan id 5`	Tạo giao diện VLAN eth0.5 trên cổng vật lý eth0 với VLAN ID = 5.
2	`ip link set eth0.5 up`	Kích hoạt giao diện VLAN để sẵn sàng truyền và nhận dữ liệu.



Sau khi thực hiện các lệnh trên, hệ thống sẽ xuất hiện giao diện mạng mới có tên **eth0.5**, đại diện cho VLAN 5 được tạo trên cổng Ethernet vật lý.

Bước 3. Cấu hình địa chỉ IPv6

Sau khi tạo giao diện VLAN, tiến hành gán địa chỉ IPv6 cho giao diện **eth0.5** bằng lệnh sau:

STT	Câu lệnh	Ý nghĩa
1	`ip -6 addr add fd53:1234:5678:1::10/64 dev eth0.5`	Gán địa chỉ IPv6 fd53:1234:5678:1::10/64 cho giao diện VLAN eth0.5. Địa chỉ này được sử dụng để giao tiếp với các thiết bị khác trong cùng mạng IPv6 thuộc VLAN 5.

Sau khi cấu hình hoàn tất, có thể kiểm tra thông tin của giao diện bằng lệnh:

```
ip -6 addr show dev eth0.5
```

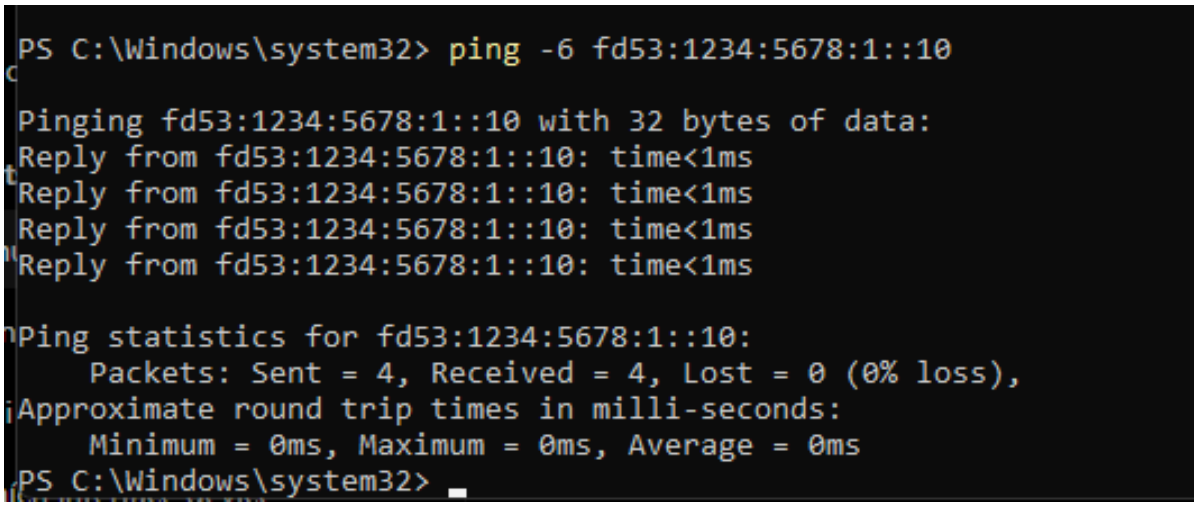
Kết quả hiển thị sẽ bao gồm địa chỉ IPv6 đã được gán cho giao diện VLAN, xác nhận rằng quá trình cấu hình đã thực hiện thành công.

Bước 4. Kiểm tra kết nối

Để xác nhận việc cấu hình IPv6 và VLAN hoạt động chính xác, tiến hành kiểm tra khả năng giao tiếp giữa máy tính Windows và Raspberry Pi bằng giao thức ICMPv6.

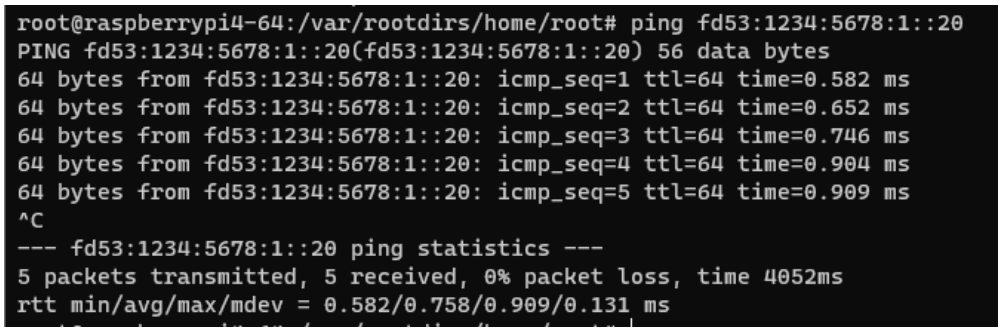
Từ máy tính Windows, gửi gói tin ICMPv6 đến địa chỉ của Raspberry Pi:

```
ping -6 fd53:1234:5678:1::10
```



Tương tự, từ Raspberry Pi có thể gửi gói tin ICMPv6 đến địa chỉ IPv6 của Windows:

```
ping -6 fd53:1234:5678:1::20
```



Nếu cả hai thiết bị đều nhận được phản hồi (Reply), điều đó chứng tỏ cấu hình VLAN và IPv6 đã chính xác, đồng thời hai thiết bị có thể giao tiếp thành công trong cùng một mạng VLAN. Đây cũng là điều kiện cần để triển khai các chương trình Python sử dụng thư viện **Scapy** nhằm tạo và gửi các gói tin IPv6 tùy chỉnh phục vụ quá trình kiểm thử và đánh giá hoạt động của hệ thống tường lửa trên Raspberry Pi.

5.2. Chương trình

Nhóm xây dựng hệ thống gồm hai chương trình Python để mô phỏng quá trình giao tiếp giữa hai thiết bị trong mạng:

- **sender.py** được thực thi trên máy tính Windows, có nhiệm vụ tạo và gửi các yêu cầu điều khiển đến Raspberry Pi.
- **server.py** được thực thi trên Raspberry Pi, có nhiệm vụ tiếp nhận yêu cầu, xử lý các thao tác cấu hình giao diện mạng và phản hồi kết quả về máy gửi.

Chương trình hỗ trợ các chức năng như:

- Hiển thị thông tin giao diện mạng.
- Thay đổi địa chỉ IPv4.
- Thay đổi địa chỉ IPv6.
- Thay đổi địa chỉ MAC.
- Thay đổi VLAN ID.
- Thực hiện các kịch bản kiểm thử bằng Scapy với nhiều loại gói tin khác nhau nhằm đánh giá cơ chế xử lý của hệ thống tường lửa.

6. Triển khai và đánh giá kết quả

6.1. Kịch bản 1 – Kiểm tra kết nối và hiển thị thông tin giao diện mạng

Mục tiêu

Kịch bản đầu tiên được xây dựng nhằm kiểm tra khả năng giao tiếp giữa máy tính Windows và Raspberry Pi thông qua mạng IPv6 trên VLAN 5, đồng thời xác minh rằng chương trình điều khiển hoạt động chính xác trước khi tiến hành các kịch bản thay đổi cấu hình mạng và kiểm thử tường lửa.

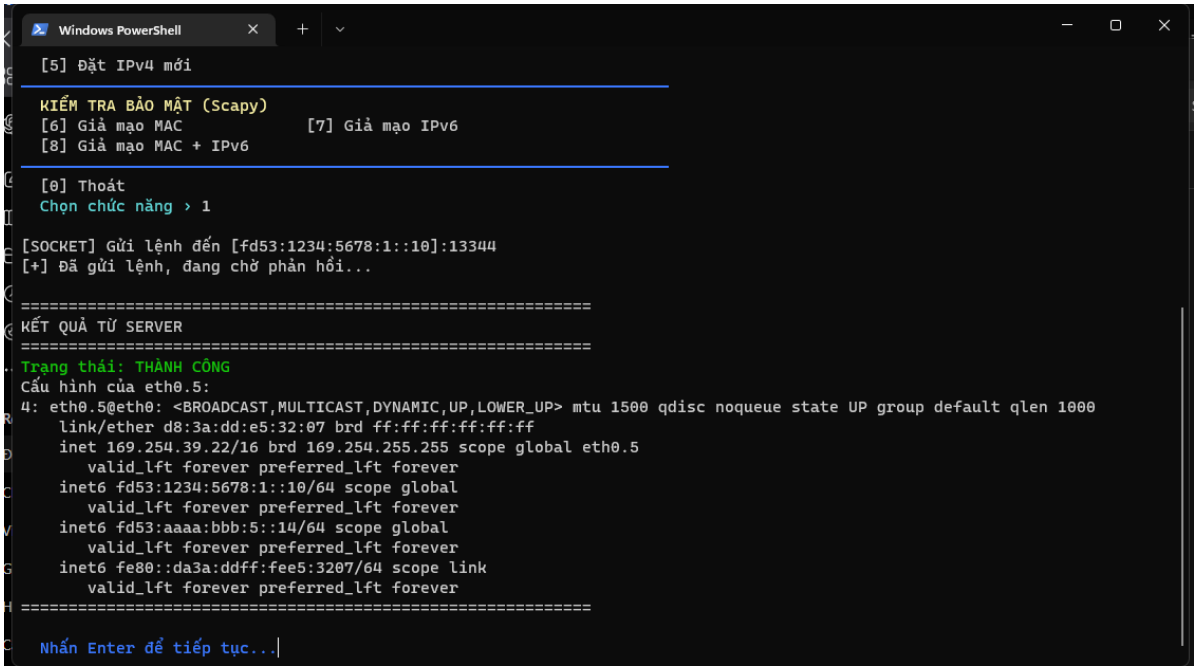
Quá trình thực hiện

Từ máy tính Windows, chương trình **sender.py** thiết lập kết nối TCP đến Raspberry Pi thông qua địa chỉ IPv6 fd53:1234:5678:1::10 tại cổng **13344**.

Người dùng lựa chọn chức năng **Hiển thị thông tin giao diện mạng**, chương trình sẽ gửi yêu cầu đến Raspberry Pi. Sau khi nhận được yêu cầu, Raspberry Pi thực thi lệnh kiểm tra cấu hình của giao diện **eth0.5** và gửi kết quả trở lại máy gửi.

Trong quá trình truyền dữ liệu, các gói tin điều khiển được gửi thông qua giao thức IPv6. Địa chỉ MAC của máy gửi nằm trong danh sách được phép (Whitelist) của tường lửa, do đó các gói tin được chấp nhận và xử lý bình thường.

Kết quả



Từ phía máy gửi, chương trình thông báo kết nối TCP được thiết lập thành công và nhận được phản hồi từ Raspberry Pi.

Thông tin trả về bao gồm:

- Trạng thái hoạt động của giao diện **eth0.5**.
- Địa chỉ MAC hiện tại của card mạng.
- Địa chỉ IPv4 đã được cấu hình.

- Địa chỉ IPv6 thuộc VLAN 5.
- Trạng thái hoạt động của giao diện mạng.

Kết quả này cho thấy Raspberry Pi đã nhận được yêu cầu từ máy gửi, xử lý thành công và phản hồi đầy đủ thông tin cấu hình của giao diện mạng.

Ở phía Raspberry Pi, hệ thống **journalctl** đồng thời ghi nhận log của nhân Linux với nội dung cho thấy gói tin điều khiển đã được tường lửa chấp nhận trên giao diện **eth0.5**. Điều này xác nhận rằng quá trình truyền thông giữa hai thiết bị diễn ra đúng như thiết kế.

Đánh giá

Kịch bản đầu tiên được thực hiện thành công. Chương trình đã thiết lập được kết nối TCP giữa Windows và Raspberry Pi, đồng thời gửi và nhận dữ liệu ổn định thông qua mạng IPv6 trên VLAN 5. Thông tin cấu hình giao diện mạng được trả về đầy đủ và chính xác, chứng minh rằng cơ chế truyền lệnh và phản hồi của hệ thống hoạt động đúng.

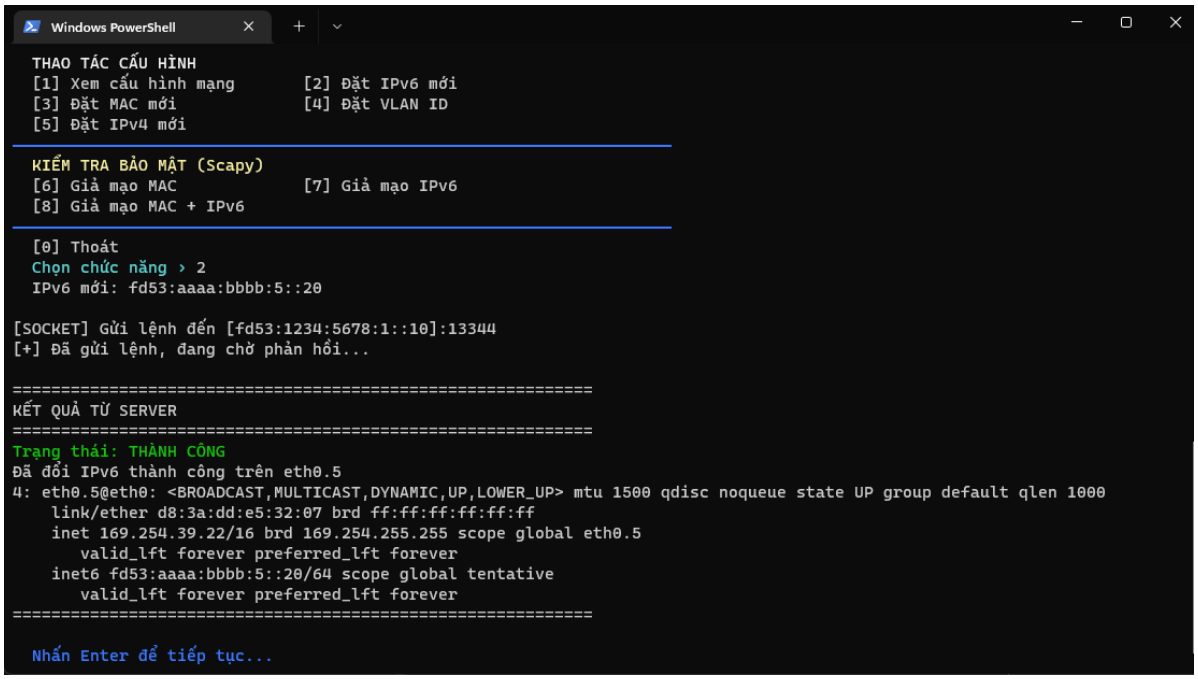
Đây là cơ sở để tiếp tục triển khai các kịch bản tiếp theo, trong đó chương trình sẽ thay đổi các trường thông tin như địa chỉ IPv4, địa chỉ IPv6, địa chỉ MAC và VLAN ID nhằm đánh giá khả năng phát hiện và xử lý của hệ thống tường lửa đối với các gói tin được tạo bằng thư viện **Scapy**.

6.1.2. Thay đổi địa chỉ IPv6

Mục tiêu

Kịch bản này nhằm kiểm tra khả năng thay đổi địa chỉ IPv6 của giao diện **eth0.5** trên Raspberry Pi thông qua chương trình điều khiển chạy trên máy tính Windows, đồng thời xác minh rằng hệ thống tường lửa chỉ cho phép các gói tin điều khiển hợp lệ được thực thi.

Quá trình thực hiện



Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Đặt IPv6 mới** và nhập địa chỉ IPv6 cần cấu hình là:

```
fd53:aaaa:bbbb:5::20
```

Chương trình thiết lập kết nối TCP đến Raspberry Pi thông qua địa chỉ fd53:1234:5678:1::10 tại cổng **13344**, sau đó gửi yêu cầu thay đổi địa chỉ IPv6.

Sau khi nhận được yêu cầu, Raspberry Pi kiểm tra các điều kiện của tường lửa. Gói tin điều khiển được chấp nhận do địa chỉ MAC nguồn và địa chỉ IPv6 nguồn đều thuộc danh sách được phép (Whitelist). Server tiếp tục xử lý yêu cầu bằng cách thực hiện các lệnh cấu hình trên hệ điều hành Linux, bao gồm:

1. Xóa địa chỉ IPv6 hiện tại của giao diện **eth0.5**.
2. Gán địa chỉ IPv6 mới fd53:aaaa:bbbb:5::20/64.
3. Kiểm tra lại cấu hình của giao diện sau khi thay đổi.

Kết quả



Trước khi thực hiện thay đổi, giao diện **eth0.5** được cấu hình với địa chỉ IPv6:

```
fd53:aaaa:bbbb:5::14/64
```

Sau khi chương trình xử lý hoàn tất, kết quả kiểm tra cho thấy địa chỉ IPv6 đã được cập nhật thành:

```
fd53:aaaa:bbbb:5::20/64
```

Trong quá trình này:

- Địa chỉ IPv6 được thay đổi thành công.
- Địa chỉ IPv4 của giao diện vẫn giữ nguyên.
- Địa chỉ MAC của card mạng không thay đổi.
- Kết nối TCP giữa máy gửi và Raspberry Pi vẫn được duy trì ổn định.

Bên cạnh đó, hệ thống **journalctl** trên Raspberry Pi ghi nhận gói tin điều khiển đã được tường lửa chấp nhận trên giao diện **eth0.5**, xác nhận rằng yêu cầu thay đổi cấu hình được thực hiện thông qua các gói tin hợp lệ.

Đánh giá

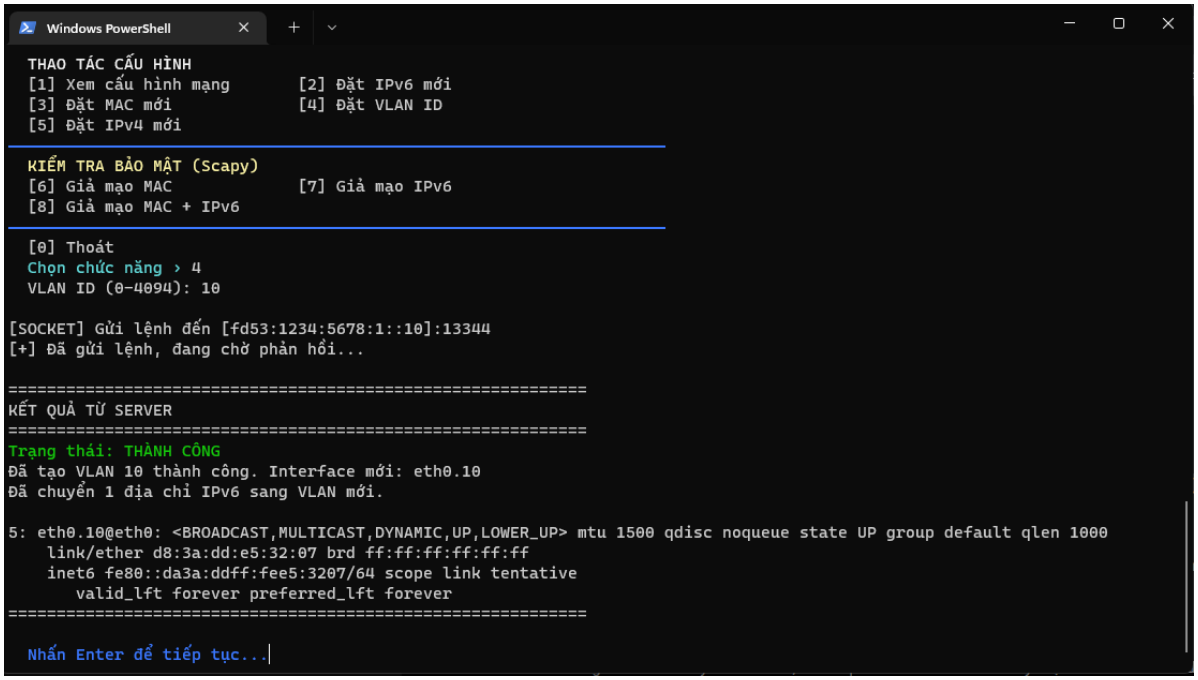
Kết quả thử nghiệm cho thấy chương trình có khả năng thay đổi địa chỉ IPv6 của Raspberry Pi từ xa thông qua kết nối TCP. Hệ thống tường lửa đã hoạt động đúng theo thiết kế khi chỉ cho phép các gói tin có địa chỉ MAC và địa chỉ IPv6 nguồn hợp lệ được thực thi. Sau khi thay đổi, giao diện mạng hoạt động bình thường và các thông số khác như địa chỉ IPv4 và địa chỉ MAC không bị ảnh hưởng, chứng tỏ quá trình cập nhật chỉ tác động đến trường địa chỉ IPv6 theo yêu cầu.

6.1.3. Thay đổi VLAN ID

Mục tiêu

Kịch bản này được thực hiện nhằm kiểm tra khả năng cấu hình VLAN từ xa trên Raspberry Pi thông qua chương trình điều khiển. Mục tiêu là xác minh hệ thống có thể tạo một giao diện VLAN mới, gán VLAN ID theo yêu cầu và chuyển địa chỉ IPv6 sang giao diện mới, đồng thời đánh giá cơ chế kiểm soát của tường lửa đối với các gói tin điều khiển.

Quá trình thực hiện



Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Đặt VLAN ID** và nhập giá trị **10**, sau đó chương trình thiết lập kết nối TCP đến Raspberry Pi và gửi yêu cầu thay đổi cấu hình VLAN.

Sau khi nhận được yêu cầu, Raspberry Pi kiểm tra thông tin của gói tin điều khiển. Do địa chỉ MAC và địa chỉ IPv6 nguồn đều nằm trong danh sách được phép (Whitelist), tường lửa chấp nhận gói tin và chuyển yêu cầu đến chương trình xử lý.

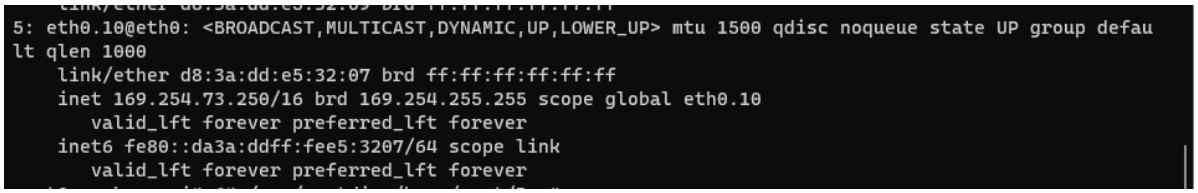
Server lần lượt thực hiện các thao tác sau:

1. Tạo giao diện VLAN mới **eth0.10** trên cổng Ethernet **eth0** với **VLAN ID = 10**.
2. Kích hoạt giao diện **eth0.10**.
3. Gán địa chỉ IPv6 hiện có cho giao diện VLAN mới.
4. Kiểm tra và trả về thông tin cấu hình sau khi hoàn tất.

Kết quả

Trước khi thực hiện kịch bản, Raspberry Pi chưa tồn tại giao diện **eth0.10**.

Sau khi chương trình xử lý thành công, hệ thống xuất hiện giao diện mạng mới **eth0.10**, hoạt động với **VLAN ID = 10**. Địa chỉ IPv6 của thiết bị đã được gán cho giao diện này và sẵn sàng sử dụng để giao tiếp trong mạng VLAN mới.



Kết quả kiểm tra cho thấy:

- Giao diện **eth0.10** được tạo thành công và ở trạng thái hoạt động (**UP**).
- Địa chỉ IPv6 được cấu hình thành công trên giao diện mới.

- Địa chỉ MAC của giao diện VLAN vẫn giữ nguyên do kế thừa từ giao diện vật lý **eth0**.
- Các giao diện khác như **eth0** và **wlan0** vẫn hoạt động bình thường và không bị ảnh hưởng.

Ngoài ra, hệ điều hành Linux tự động gán địa chỉ IPv4 **169.254.x.x/16** (Link-local) cho giao diện mới. Đây là cơ chế mặc định của hệ điều hành khi giao diện chưa được cấu hình địa chỉ IPv4 tĩnh hoặc chưa nhận được địa chỉ từ máy chủ DHCP, không ảnh hưởng đến quá trình truyền thông IPv6 trong bài thực hành.

Ở phía Raspberry Pi, hệ thống **journalctl** ghi nhận gói tin điều khiển đã được tường lửa chấp nhận và chương trình xử lý hoàn tất yêu cầu tạo VLAN, xác nhận toàn bộ quá trình diễn ra đúng theo thiết kế.

Đánh giá

Kết quả thử nghiệm cho thấy chương trình có khả năng cấu hình VLAN từ xa thông qua kết nối TCP giữa Windows và Raspberry Pi. Khi nhận được yêu cầu hợp lệ, hệ thống đã tạo thành công giao diện **eth0.10**, gán địa chỉ IPv6 cho giao diện mới và duy trì hoạt động ổn định của các giao diện mạng còn lại.

Kịch bản cũng chứng minh rằng tường lửa chỉ cho phép các gói tin điều khiển có địa chỉ MAC và địa chỉ IPv6 nguồn hợp lệ được thực thi. Điều này góp phần đảm bảo chỉ các thiết bị được ủy quyền mới có thể thay đổi cấu hình mạng của Raspberry Pi, đáp ứng yêu cầu về tính an toàn và bảo mật của hệ thống.

6.1.4. Thay đổi địa chỉ MAC

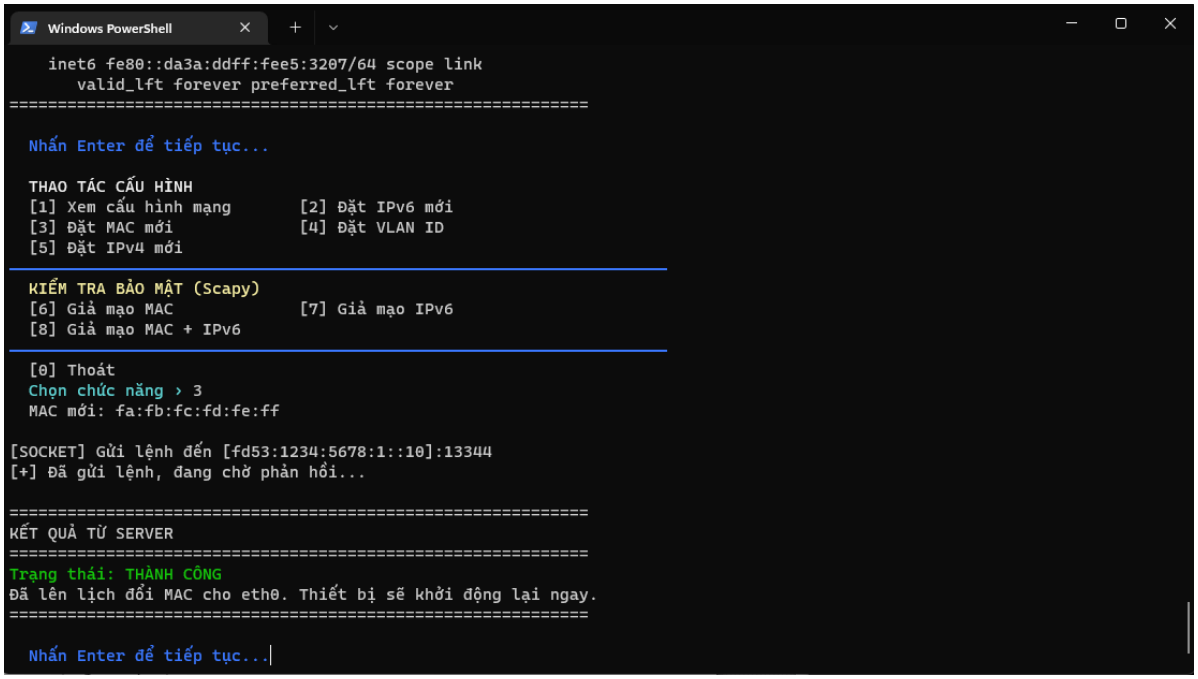
Mục tiêu

Kịch bản này được thực hiện nhằm kiểm tra khả năng thay đổi địa chỉ MAC của giao diện mạng trên Raspberry Pi thông qua chương trình điều khiển từ xa. Đồng thời, kịch bản cũng đánh giá cơ chế xác thực của tường lửa trước khi cho phép thực hiện các thao tác thay đổi cấu hình mạng.

Quá trình thực hiện

Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Thay đổi địa chỉ MAC** và nhập địa chỉ MAC mới:

```
fa:fb:fc:fd:fe:ff
```



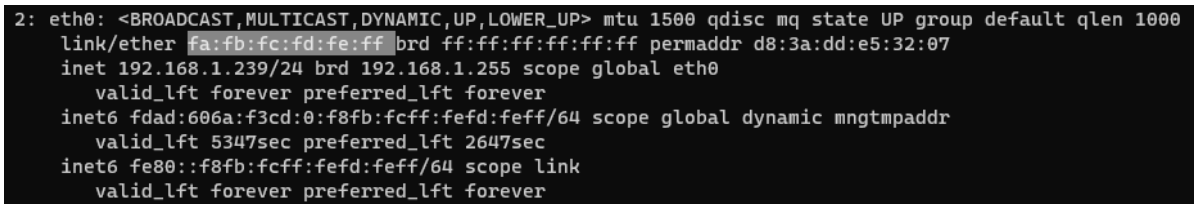
Chương trình thiết lập kết nối TCP đến Raspberry Pi và gửi yêu cầu thay đổi địa chỉ MAC. Gói tin điều khiển chứa địa chỉ MAC và địa chỉ IPv6 nguồn hợp lệ nên được tường lửa chấp nhận và chuyển đến chương trình xử lý.

Sau khi tiếp nhận yêu cầu, server thực hiện các thao tác cần thiết để thay đổi địa chỉ MAC. Do việc thay đổi địa chỉ MAC của giao diện mạng có thể làm gián đoạn kết nối hiện tại, chương trình tạo một **systemd service** để áp dụng cấu hình sau khi hệ thống khởi động lại, sau đó tự động thực hiện lệnh **reboot**.

Kết quả

Trước khi thực hiện thay đổi, giao diện **eth0** sử dụng địa chỉ MAC mặc định của thiết bị.

Sau khi server tiếp nhận yêu cầu, kết nối TCP giữa Windows và Raspberry Pi bị ngắt. Đây là hiện tượng bình thường do hệ thống khởi động lại để áp dụng cấu hình mới cho giao diện mạng.



Sau khi Raspberry Pi khởi động hoàn tất, kiểm tra cấu hình cho thấy:

- Địa chỉ MAC của giao diện **eth0** đã được thay đổi.
- Hệ thống tự động tạo lại địa chỉ IPv6 Link-local tương ứng với địa chỉ MAC mới.
- Giao diện mạng hoạt động bình thường sau khi khởi động lại.
- Các giao diện mạng khác, chẳng hạn như **wlan0**, không bị ảnh hưởng.

Đánh giá

Kết quả thử nghiệm cho thấy chương trình có khả năng thay đổi địa chỉ MAC của Raspberry Pi thông qua cơ chế điều khiển từ xa. Tường lửa chỉ cho phép yêu cầu được thực thi khi gói tin điều khiển có địa chỉ MAC và địa chỉ IPv6 nguồn hợp lệ, đảm bảo chỉ các thiết bị được ủy quyền mới có thể thay đổi cấu hình mạng.

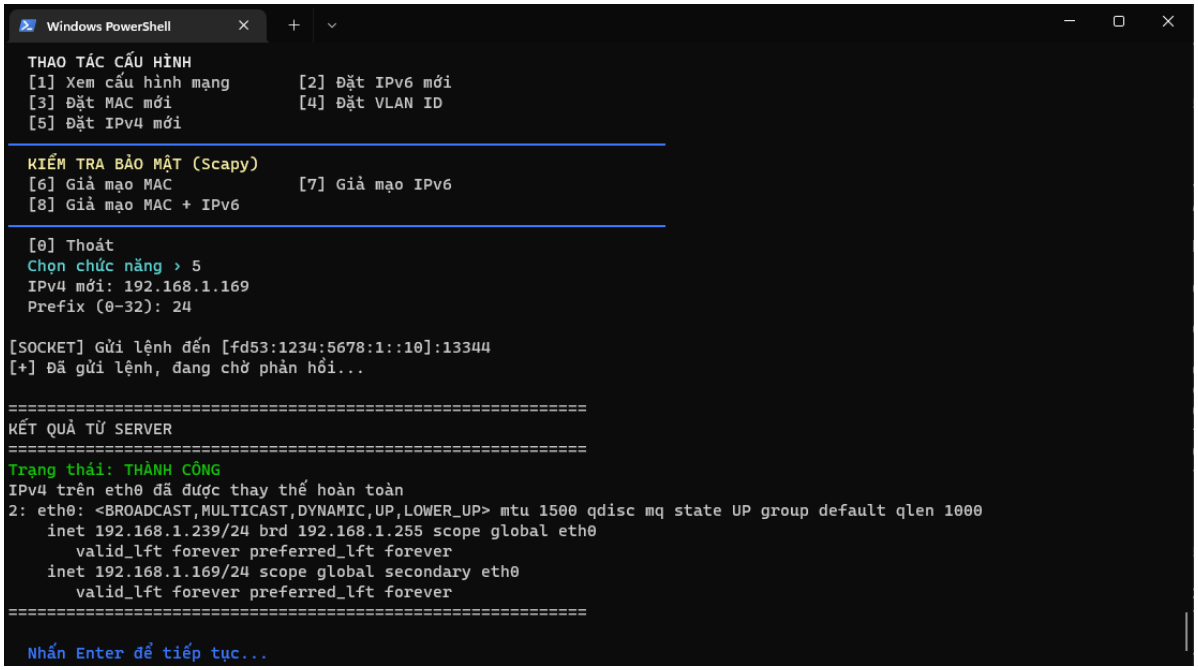
Quá trình thay đổi địa chỉ MAC yêu cầu khởi động lại hệ thống để cấu hình mới được áp dụng hoàn toàn. Sau khi khởi động lại, giao diện mạng hoạt động ổn định với địa chỉ MAC mới, chứng minh cơ chế cập nhật cấu hình thông qua **systemd service** hoạt động đúng theo thiết kế.

6.1.5. Thay đổi địa chỉ IPv4

Mục tiêu

Kịch bản này được thực hiện nhằm kiểm tra khả năng thay đổi địa chỉ IPv4 của giao diện mạng trên Raspberry Pi thông qua chương trình điều khiển từ xa. Đồng thời, kịch bản đánh giá khả năng xử lý của hệ thống khi cập nhật địa chỉ IPv4 mà không làm ảnh hưởng đến các thông số mạng khác như địa chỉ IPv6 và địa chỉ MAC.

Quá trình thực hiện



Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Đặt IPv4 mới** và nhập địa chỉ IPv4 cần cấu hình là **192.168.1.169/24**. Chương trình thiết lập kết nối TCP đến Raspberry Pi thông qua địa chỉ IPv6 và gửi yêu cầu thay đổi cấu hình.

Sau khi tiếp nhận yêu cầu, Raspberry Pi kiểm tra thông tin của gói tin điều khiển. Do địa chỉ MAC và địa chỉ IPv6 nguồn đều thuộc danh sách được phép (Whitelist), tường lửa chấp nhận gói tin và chuyển yêu cầu đến chương trình xử lý.

Server lần lượt thực hiện các thao tác sau:

1. Xóa toàn bộ địa chỉ IPv4 hiện có trên giao diện **eth0**.
2. Gán địa chỉ IPv4 mới **192.168.1.169/24** cho giao diện **eth0**.

3. Kiểm tra và trả về thông tin cấu hình của giao diện sau khi cập nhật.

Kết quả

Trước khi thực hiện kịch bản, giao diện **eth0** được cấu hình với địa chỉ IPv4 hiện có của hệ thống.

Sau khi chương trình xử lý hoàn tất, kết quả kiểm tra cho thấy địa chỉ IPv4 của giao diện **eth0** đã được cập nhật thành **192.168.1.169/24**.

Quá trình thay đổi cấu hình cho thấy:

- Địa chỉ IPv4 được cập nhật thành công theo yêu cầu.
- Địa chỉ IPv6 của giao diện vẫn được giữ nguyên.
- Địa chỉ MAC của card mạng không thay đổi.
- Không cần khởi động lại Raspberry Pi để áp dụng cấu hình mới.

Ngoài ra, hệ thống **journalctl** ghi nhận gói tin điều khiển đã được tường lửa chấp nhận và chương trình xử lý thành công yêu cầu thay đổi địa chỉ IPv4, chứng minh quá trình truyền lệnh và thực thi diễn ra đúng như thiết kế.

Đánh giá

Kết quả thử nghiệm cho thấy chương trình có khả năng thay đổi địa chỉ IPv4 của Raspberry Pi từ xa thông qua kết nối TCP giữa máy tính Windows và Raspberry Pi. Sau khi nhận được yêu cầu hợp lệ, server đã cập nhật thành công địa chỉ IPv4 mà không làm ảnh hưởng đến địa chỉ IPv6, địa chỉ MAC cũng như trạng thái hoạt động của giao diện mạng.

Khác với kịch bản thay đổi địa chỉ MAC, việc cập nhật địa chỉ IPv4 không yêu cầu khởi động lại hệ thống. Điều này giúp giảm thời gian gián đoạn dịch vụ và cho phép cấu hình mới được áp dụng ngay sau khi lệnh được thực thi. Kết quả cũng cho thấy cơ chế kiểm soát của tường lửa hoạt động đúng theo thiết kế khi chỉ cho phép các gói tin điều khiển có địa chỉ MAC và địa chỉ IPv6 nguồn hợp lệ được thực hiện.

6.2. Kịch bản 2 – Kiểm tra cơ chế chặn của tường lửa

Khác với kịch bản 1, trong kịch bản này nhóm tiến hành tạo các gói tin có thông tin nhận dạng không hợp lệ nhằm đánh giá khả năng kiểm soát truy cập của hệ thống tường lửa. Cụ thể, các trường như địa chỉ MAC hoặc địa chỉ IPv6 nguồn được thay đổi sao cho không nằm trong danh sách **Whitelist** đã được cấu hình trước. Mục tiêu là xác minh rằng tường lửa có thể phát hiện và loại bỏ các gói tin không hợp lệ trước khi chúng được chuyển đến chương trình điều khiển trên Raspberry Pi.

6.2.1. Kiểm tra với địa chỉ MAC không hợp lệ

Mục tiêu

Kịch bản này nhằm đánh giá trực tiếp tác động của rule kiểm tra địa chỉ MAC bằng cách gửi cùng một gói tin giả mạo trong hai trạng thái: Raspberry Pi có áp dụng rule firewall và Raspberry Pi không áp dụng rule. Qua đó xác định việc kết nối thất bại là do cơ chế lọc của tường lửa, không phải do lỗi đường truyền hoặc dịch vụ TCP.

Quá trình thực hiện

Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Kiểm tra giả mạo địa chỉ MAC**. Scapy tạo gói tin TCP với các thông tin sau:

- Địa chỉ MAC nguồn giả mạo: `A2:B3:C4:E5:F4:A7`.
- Địa chỉ IPv6 nguồn hợp lệ của laptop: `fd53:1234:5678:1::20`.
- Địa chỉ IPv6 đích của Raspberry Pi: `fd53:1234:5678:1::10`.
- Cổng dịch vụ đích: **13344**.

Trường hợp 1: Raspberry Pi có áp dụng rule firewall.

Trên Raspberry Pi, bảng quy tắc firewall chỉ cho phép kết nối đến cổng **13344** từ địa chỉ MAC hợp lệ **50:EB:F6:D2:67:3E**; các gói tin TCP còn lại gửi đến cổng này sẽ bị loại bỏ. Trạng thái rule được kiểm tra trước khi thực hiện thử nghiệm.

```
root@raspberrypi4-64:/var/rootdirs/home/root/Bao# ip6tables -nvL
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     tcp      eth0   *       ::/0                 fd53:1234:5678:1::10  tcp dpt:13344 MAC50:eb:f6:d2:67:3e
    1   193 DROP       tcp      eth0   *       ::/0                 fd53:1234:5678:1::10  tcp dpt:13344

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
```

Khi gửi gói tin mang địa chỉ MAC giả mạo, chương trình chờ phản hồi trong ba giây nhưng không nhận được gói **SYN-ACK**. Quá trình bắt tay TCP thất bại và chương trình xác định gói tin đã bị firewall chặn.

```
Administrator: Windows Pow

  THAO TÁC CẤU HÌNH
  [1] Xem cấu hình mạng       [2] Đặt IPv6 mới
  [3] Đặt MAC mới             [4] Đặt VLAN ID
  [5] Đặt IPv4 mới

  KIỂM TRA BẢO MẬT (Scapy)
  [6] Giả mạo MAC             [7] Giả mạo IPv6
  [8] Giả mạo MAC + IPv6

  [0] Thoát
  Chọn chức năng › 6

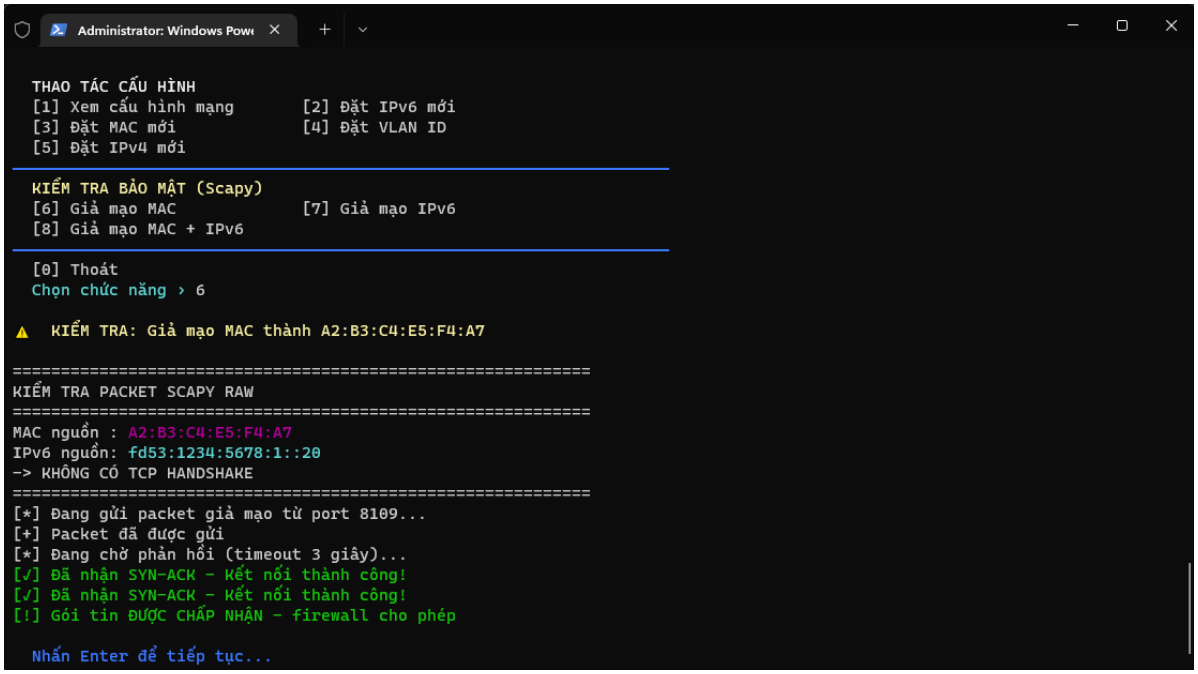
▲  KIỂM TRA: Giả mạo MAC thành A2:B3:C4:E5:F4:A7

============================================================
KIỂM TRA PACKET SCAPY RAW
============================================================
MAC nguồn : A2:B3:C4:E5:F4:A7
IPv6 nguồn: fd53:1234:5678:1::20
-> KHÔNG CÓ TCP HANDSHAKE
============================================================
[*] Đang gửi packet giả mạo từ port 63981...
[+] Packet đã được gửi
[*] Đang chờ phản hồi (timeout 3 giây)...
[LỖI] Bắt tay thất bại: Không nhận được SYN-ACK.
>>> KIỂM TRA: Gửi từ MAC giả mạo. Điều này nên thất bại.
--- SRC_MAC: A2:B3:C4:E5:F4:A7, SRC_IPv6: fd53:1234:5678:1::20 ---
[!] Gói tin có thể đã BỊ DROP bởi nftables/firewall

  Nhấn Enter để tiếp tục...
```

Trường hợp 2: Raspberry Pi không áp dụng rule firewall.

Rule kiểm tra MAC được gỡ bỏ hoặc tạm thời vô hiệu hóa, sau đó chương trình gửi lại gói tin với cùng địa chỉ MAC giả mạo, địa chỉ IPv6 nguồn và cổng đích. Lần thử này nhận được phản hồi **SYN-ACK**, quá trình bắt tay TCP hoàn tất và kết nối được thiết lập thành công.



Kết quả

- **Khi có rule firewall:** gói tin giả MAC không nhận được phản hồi SYN-ACK, hết thời gian chờ và không thể thiết lập kết nối TCP.
- **Khi không có rule firewall:** cùng gói tin giả MAC nhận được phản hồi SYN-ACK và thiết lập kết nối TCP thành công với Raspberry Pi.

Sự khác biệt giữa hai lần thử cho thấy kết nối mạng IPv6 và dịch vụ tại cổng **13344** vẫn hoạt động bình thường; nguyên nhân làm lần thử đầu tiên thất bại chính là rule kiểm tra địa chỉ MAC.

Đánh giá

Phép thử đối chứng xác nhận rule **Whitelist địa chỉ MAC** hoạt động đúng theo thiết kế. Khi rule được áp dụng, gói tin có IPv6 và cổng đích hợp lệ vẫn bị loại bỏ nếu địa chỉ MAC nguồn không nằm trong danh sách cho phép. Khi rule được gỡ bỏ, gói tin tương tự có thể hoàn tất bắt tay TCP.

Kết quả chứng minh cơ chế lọc theo địa chỉ MAC có khả năng ngăn thiết bị sử dụng thông tin nhận dạng không hợp lệ truy cập dịch vụ trên Raspberry Pi, đồng thời xác nhận việc chặn gói tin không xuất phát từ lõi kết nối mạng.

6.2.2. Kiểm tra với địa chỉ IPv6 không hợp lệ

Mục tiêu

Kịch bản này nhằm đánh giá khả năng của tường lửa trong việc phát hiện và chặn gói tin có địa chỉ IPv6 nguồn không thuộc danh sách **Whitelist**, trong khi địa chỉ MAC nguồn vẫn hợp lệ. Để xác định chính xác tác động của rule lọc IPv6, cùng một gói tin giả mạo được kiểm thử trong hai trường hợp: Raspberry Pi có áp dụng rule firewall và Raspberry Pi không áp dụng rule firewall.

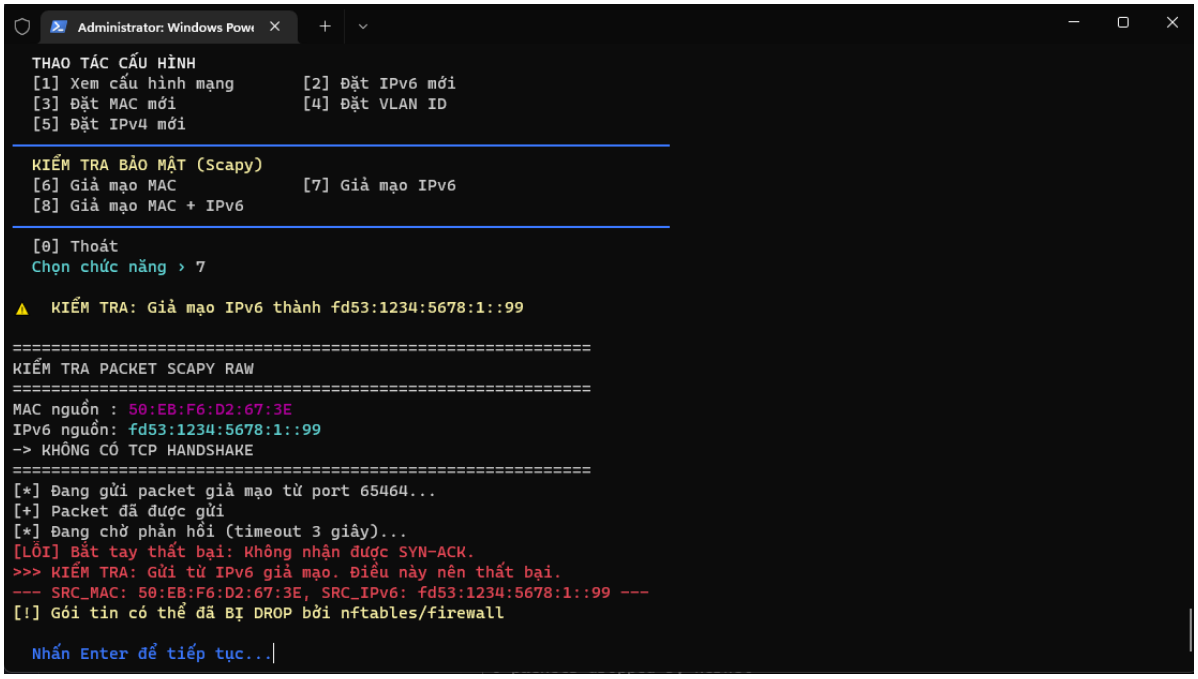
Quá trình thực hiện

Từ chương trình **sender.py** trên máy tính Windows, người dùng lựa chọn chức năng **Kiểm tra giả mạo địa chỉ IPv6**. Chương trình sử dụng thư viện **Scapy** để tạo và gửi gói tin TCP đến cổng dịch vụ **13344** của Raspberry Pi với các thông tin chính như sau:

- Địa chỉ MAC nguồn hợp lệ: **50:EB:F6:D2:67:3E**.
- Địa chỉ IPv6 nguồn giả mạo: **fd53:1234:5678:1::99**.

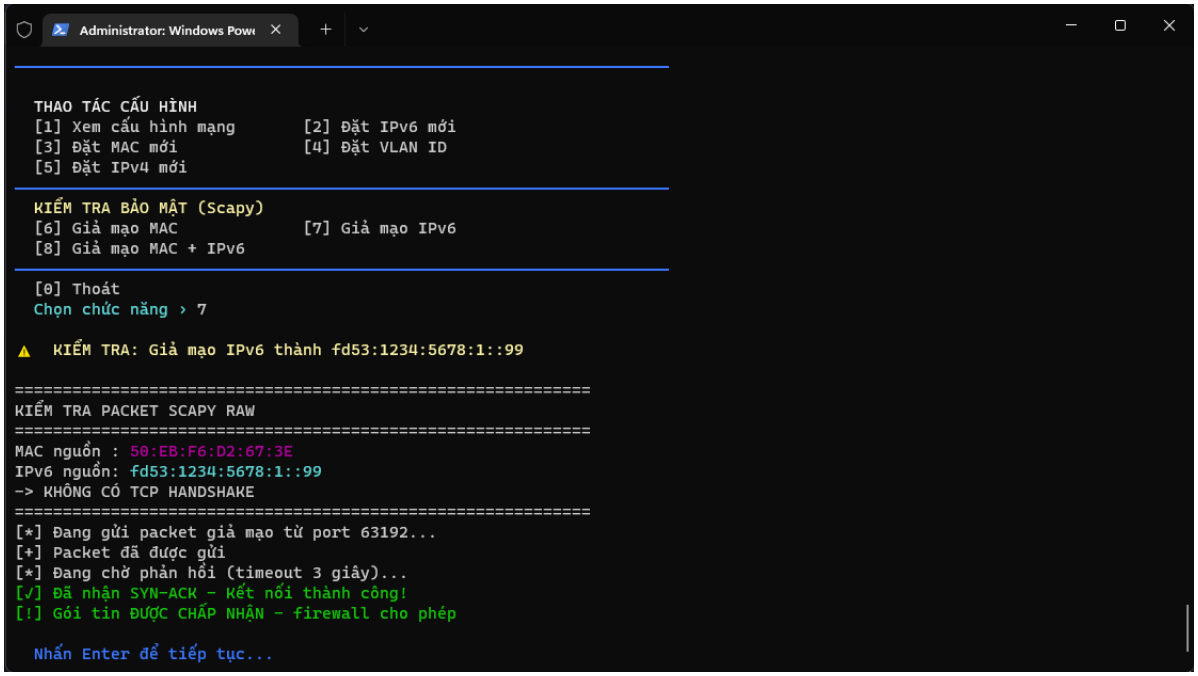
Trường hợp 1: Raspberry Pi có áp dụng rule firewall.

Khi rule kiểm tra địa chỉ IPv6 đang hoạt động, địa chỉ nguồn giả mạo không thuộc danh sách được phép. Chương trình gửi gói tin và chờ phản hồi trong ba giây nhưng không nhận được **SYN-ACK**; bước kiểm tra kết nối TCP vì vậy được xác định là thất bại.



Trường hợp 2: Raspberry Pi không áp dụng rule firewall.

Rule kiểm tra địa chỉ IPv6 được gỡ bỏ hoặc tạm thời vô hiệu hóa, sau đó chương trình gửi lại gói tin với cùng địa chỉ MAC, địa chỉ IPv6 giả mạo và cổng đích. Trong lần thử này, máy gửi nhận được phản hồi **SYN-ACK** và chương trình thông báo kết nối thành công.



Kết quả

- **Khi có rule firewall:** gói tin giả IPv6 không nhận được phản hồi SYN-ACK, hết thời gian chờ và không thể thiết lập kết nối TCP.
- **Khi không có rule firewall:** cùng gói tin giả IPv6 nhận được phản hồi SYN-ACK và chương trình xác định kết nối đến Raspberry Pi thành công.

Sự khác biệt giữa hai lần thử cho thấy địa chỉ MAC, đường truyền IPv6 và dịch vụ tại cổng **13344** vẫn hoạt động bình thường. Nguyên nhân khiến lần thử đầu tiên thất bại là rule kiểm tra địa chỉ IPv6 nguồn trên Raspberry Pi.

Đánh giá

Phép thử đối chứng xác nhận rule **Whitelist địa chỉ IPv6** hoạt động đúng theo thiết kế. Khi rule được áp dụng, gói tin có địa chỉ MAC hợp lệ và được gửi đến đúng cổng dịch vụ vẫn bị loại bỏ nếu địa chỉ IPv6 nguồn không nằm trong danh sách cho phép. Khi rule được gỡ bỏ, gói tin tương tự có thể nhận được phản hồi từ Raspberry Pi.

Kết quả chứng minh cơ chế lọc theo địa chỉ IPv6 có khả năng ngăn chặn các gói tin sử dụng địa chỉ nguồn giả mạo, đồng thời xác nhận việc mất kết nối trong lần thử đầu tiên không xuất phát từ lỗi mạng hoặc dịch vụ.

6.2.3. Kiểm tra với địa chỉ MAC và IPv6 đều không hợp lệ

Mục tiêu

Kịch bản này được thực hiện nhằm đánh giá khả năng của tường lửa khi gói tin đồng thời sử dụng địa chỉ MAC và địa chỉ IPv6 nguồn không thuộc danh sách **Whitelist**. Qua đó xác minh cơ chế xử lý ưu tiên của hệ thống đối với nhiều điều kiện kiểm tra bảo mật.

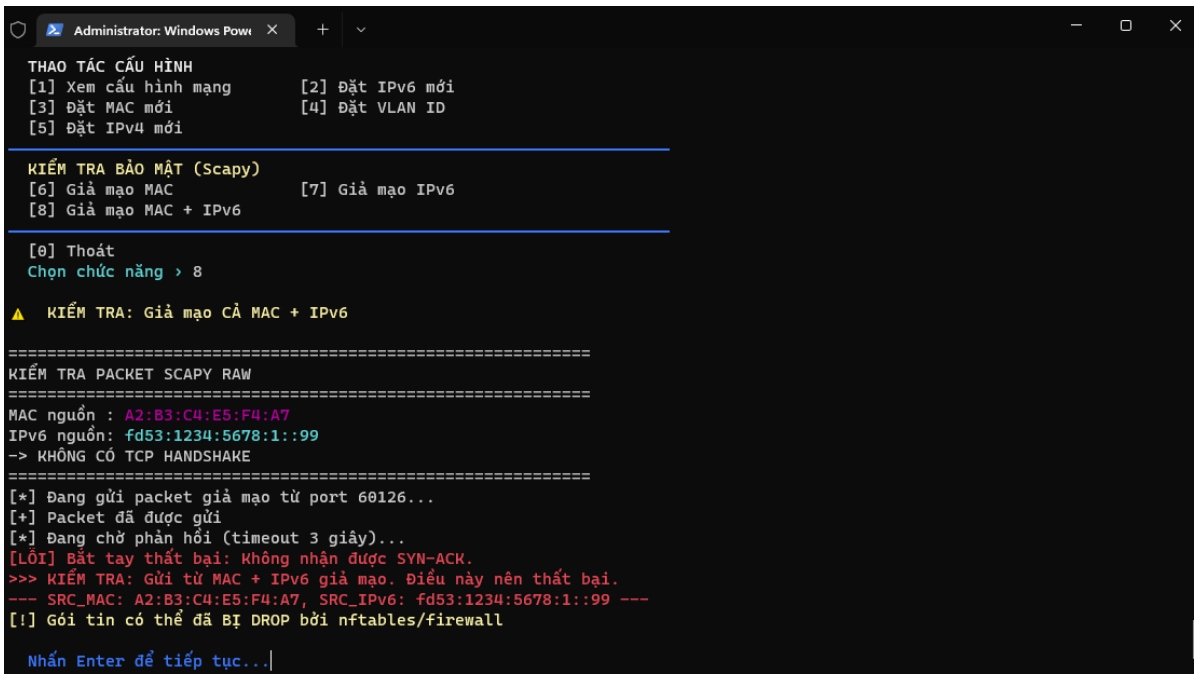
Quá trình thực hiện

Từ chương trình **sender.py**, người dùng lựa chọn chức năng **Kiểm tra giả mạo địa chỉ MAC và IPv6**. Chương trình sử dụng **Scapy** để tạo gói tin TCP với:

- Địa chỉ MAC nguồn giả mạo.
- Địa chỉ IPv6 nguồn giả mạo.

Sau đó gói tin được gửi đến Raspberry Pi thông qua cổng **13344** để kiểm tra phản ứng của hệ thống.

Kết quả



Trong quá trình gửi, máy tính Windows không nhận được phản hồi **SYN-ACK**, chứng tỏ kết nối TCP không được thiết lập.

Tại Raspberry Pi, hệ thống **journalctl** ghi nhận gói tin đến nhưng tường lửa đã loại bỏ ngay do địa chỉ MAC nguồn không hợp lệ. Vì gói tin đã bị chặn ở bước kiểm tra đầu tiên nên chương trình **receiver.py** không nhận được bất kỳ yêu cầu nào từ máy gửi.

Đánh giá

Kết quả thử nghiệm cho thấy khi cả địa chỉ MAC và địa chỉ IPv6 đều không hợp lệ, tường lửa sẽ loại bỏ gói tin ngay từ bước kiểm tra địa chỉ MAC mà không cần tiếp tục xử lý các điều kiện còn lại. Điều này giúp giảm chi phí xử lý và hạn chế nguy cơ các gói tin độc hại tiếp cận chương trình ứng dụng.

Kết quả cũng chứng minh cơ chế **Whitelist** của hệ thống hoạt động hiệu quả khi chỉ cho phép các thiết bị có đầy đủ thông tin nhận dạng hợp lệ được thiết lập kết nối với Raspberry Pi.

7. Kết luận

Qua quá trình xây dựng hệ thống và thực hiện các kịch bản kiểm thử, nhóm đã đánh giá được khả năng bảo vệ của tường lửa **nftables** đối với các gói tin điều khiển được tạo bằng thư viện **Scapy**. Các thử nghiệm bao gồm cả trường hợp gói tin hợp lệ và các trường hợp giả mạo thông tin nhận dạng như địa chỉ MAC, địa chỉ IPv6 hoặc đồng thời giả mạo cả hai trường thông tin.

Kết quả thực nghiệm cho thấy hệ thống hoạt động đúng theo thiết kế. Các gói tin có thông tin nhận dạng hợp lệ được tường lửa cho phép đi qua và chương trình trên Raspberry Pi xử lý thành công. Ngược lại, các gói tin có địa chỉ MAC hoặc địa chỉ IPv6 không thuộc danh sách **Whitelist** đều bị tường lửa loại bỏ, khiến kết nối TCP không thể được thiết lập và chương trình điều khiển không nhận được yêu cầu từ máy gửi. Đồng thời, hệ thống **journalctl** ghi nhận đầy đủ các sự kiện liên quan đến việc cho phép hoặc từ chối gói tin, hỗ trợ hiệu quả cho quá trình theo dõi, phân tích và đánh giá kết quả kiểm thử.

Qua các kịch bản thực nghiệm có thể thấy cơ chế **Whitelist** đóng vai trò quan trọng trong việc kiểm soát truy cập vào hệ thống. Địa chỉ MAC được sử dụng như điều kiện xác thực đầu tiên nhằm loại bỏ các thiết bị không được phép truy cập, trong khi địa chỉ IPv6 tiếp tục được sử dụng để xác minh danh tính của thiết bị trước khi chương trình xử lý yêu cầu điều khiển. Việc kết hợp nhiều tiêu chí xác thực giúp nâng cao khả năng bảo vệ hệ thống trước các hành vi giả mạo và truy cập trái phép.

Nhìn chung, đề tài đã hoàn thành mục tiêu đặt ra là xây dựng thành công mô hình điều khiển và kiểm thử hệ thống mạng sử dụng **Scapy**, **IPv6**, **VLAN** và **nftables** trên nền tảng Raspberry Pi. Kết quả thực nghiệm chứng minh mô hình hoạt động ổn định, đáp ứng yêu cầu về tính chính xác, an toàn và khả năng mở rộng, đồng thời có thể làm nền tảng cho các nghiên cứu tiếp theo về kiểm thử bảo mật mạng, phát triển cơ chế phát hiện tấn công hoặc xây dựng các giải pháp quản lý và giám sát hệ thống mạng thông minh.